

LAPORAN PRAKTIKUM

▪ Identitas Praktikum

Nama MK : Praktikum Pemrograman Berorientasi Objek
Kode MK : CAK2KAB4
Bobot SKS : 3 SKS
Tempat : L-MM, Gedung IOT, lantai 3
Hari, tanggal : Selasa, 28 April 2026
Jam : 09:30-12:30 WIB
Topik praktikum : Modul-9

▪ Identitas Mahasiswa

Nama lengkap : Keishin Naufa Alfaridzhi
NIM : 103112400061
Program Studi : S-1 Teknik Informatika

▪ Kode Program & Penjelasan

Variable Casting

(DownCast.java)

```
package Modul9.VariableCasting;

class A {}
class B extends A {}

public class DownCast {
    public static void main(String[] args) {
        DownCast t = new DownCast();
        t.lakukanSesuatu();
    }
    void lakukanSesuatu() {
        A a = new A();
        /*Statement di bawah dapat di compile, akan tetapi
        *saat dijalankan akan melemparkan exception
        *ClassCastException !!!!
        */
        B b = (B) a;
    }
}

/*
Saat di compile berhasil !!! (dengan javac)
Saat dijalankan (dengan java) akan menghasilkan :
java.lang.NoClassDefFoundError: inheritance/VarRefCast03
```

```
Exception in thread "main"
Java Result: 1
*/

public interface Pembayaran {
    void bayar(double jumlah);
    void cekSaldo();
    List<String> getRiwayat();
}
```

Penjelasan:

Class **DownCast** digunakan untuk mendemonstrasikan proses downcasting. Untuk penjelasan terkait class ini adalah sebagai berikut:

Method `IAkukanSesuatu()` Melakukan instansiasi objek dari superclass A, kemudian mencoba melakukan downcasting secara eksplisit ke subclass B menggunakan `(B) a`. Statement ini dapat dikompilasi tanpa error, namun saat dijalankan akan menghasilkan `ClassCastException` karena objek aslinya bukan merupakan instansiasi dari subclass B.

(UpCast.java)

```
package MODUL67.Abstract;

import java.util.ArrayList;
import java.util.List;

public abstract class MetodePembayaran implements Pembayaran {
    private String namaMetode;
    private double saldo;
    private List<String> riwayat;

    public MetodePembayaran(String namaMetode, double saldoAwal) {
        this.namaMetode = namaMetode;
        this.saldo = saldoAwal;
        this.riwayat = new ArrayList<>();
    }

    public abstract boolean validasiPembayaran(double jumlah);

    public String getNamaMetode() {
        return namaMetode;
    }

    public double getSaldo() {
        return saldo;
    }

    public void setSaldo(double saldo) {
        this.saldo = saldo;
    }

    public void tambahRiwayat(String catatan) {
        riwayat.add(catatan);
    }
}
```

```

@Override
public void cekSaldo() {
    System.out.println(namaMetode + " -> Rp " + (Long) saldo);
}

@Override
public List<String> getRiwayat() {
    return riwayat;
}
} package Modul9.VariableCasting;

class A {}
class B extends A {}

public class UpCast {
    public static void main(String[] args) {
        B b = new B();
        //implisit upcasting
        //dari variabel reference ke variabel reference lainnya
        A ab1 = b;
        //implisit upcasting
        //dari suatu Object ke variabel reference
        A ab2 = new B();
        //eksplisit upcasting
        //dari variabel reference ke variabel reference lainnya
        A ab3 = (A) b;
        //eksplisit upcasting
        //dari Object ke variabel reference
        A ab4 = (A) new B();
    }
}

```

Penjelasan:

Class **UpCast** digunakan untuk mendemonstrasikan proses upcasting. Untuk penjelasan terkait class ini adalah sebagai berikut:

Method main() Melakukan instansiasi objek dari subclass B. Program kemudian mendemonstrasikan proses upcasting, yaitu mengubah reference dari subclass ke superclass A. Upcasting dilakukan secara implisit dan eksplisit, di mana operasi ini berjalan dengan aman tanpa menghasilkan compilation error maupun runtime error.

Static

(StaticVariable.java)

```

package Modul9.Static;

class Share{
    private int privateInt;
    private static int staticInt;
    public Share(int pr, int si){
        privateInt = pr;
        staticInt = si;
    }
}

```

```

public String toString(){
    return privateInt + "" + staticInt;
}
}

class StaticVariable{
    public static void main(String args[]){
        Share s1 = new Share(4,4);
        System.out.println(s1.toString());
        Share s2 = new Share(8,2);
        System.out.println(s1.toString());
        System.out.println(s2.toString());
        Share s3 = new Share(6,22);
        System.out.println(s1.toString());
        System.out.println(s2.toString());
        System.out.println(s3.toString());
    }
}

```

Penjelasan:

StaticVariable.java mendemonstrasikan penggunaan variabel static melalui class **Share** dan class utama **StaticVariable**. Untuk penjelasan adalah sebagai berikut:

1. Atribut pada class **Share**

Terdapat atribut `privateInt` bertipe `int` sebagai variabel instance yang bersifat independen, dan `staticInt` bertipe `int` sebagai variabel class yang nilainya dibagikan secara global ke seluruh objek.

2. Constructor pada class **Share**

Menerima parameter **pr** dan **si** untuk diinisialisasi ke atribut class. Karena `staticInt` bersifat static, setiap kali constructor dipanggil, perubahannya berdampak secara langsung pada seluruh objek `Share` yang pernah dibuat.

3. Method **toString()** pada class `Share`

Mengembalikan representasi string yang menggabungkan nilai `privateInt` dan `staticInt` agar dapat di-print.

4. Method **main()** pada class **StaticVariable**

Melakukan instansiasi tiga objek `Share` secara berurutan. Saat dicetak, nilai `staticInt` pada ketiga objek selalu menampilkan angka yang sama mengikuti input yang terakhir kali dimasukkan, sementara `privateInt` mempertahankan nilai awalnya masing-masing.

(StaticMethod.java)

```

package Modul9.Static;

class CounterMachine{
    static int counter;
    public static void count() {
        counter++;
    }
}

```

```

    }
}

public class StaticMethod {
    public static void main(String args[]){
        for(int i = 0; i < 10; i++){
            if( i % 2 == 0){
                CounterMachine.count();
            }
        }
        System.out.println(CounterMachine.counter);
    }
}

```

Penjelasan:

StaticMethod.java mendemonstrasikan penggunaan fungsi static melalui class **CounterMachine** dan class utama **StaticMethod**. Untuk penjelasan rinci terkait file ini adalah sebagai berikut:

1. Atribut pada class **CounterMachine**

Terdapat atribut counter bertipe int dengan modifier static yang berfungsi sebagai tempat penyimpanan hitungan secara global.

2. Method **count()** pada class CounterMachine

Merupakan static method yang berisi logika matematika sederhana untuk melakukan increment (counter++). Method ini dapat dieksekusi secara langsung menggunakan nama class tanpa perlu melakukan instansiasi objek.

3. Method **main()** pada class StaticMethod

Menjalankan perulangan for dari 0 hingga 9 dan menyaring nilai genap menggunakan kondisional if. Setiap kali kondisi bernilai true, method CounterMachine.count() dipanggil secara langsung untuk menambah nilai counter.

Collection

(Collection.java)

```

package Modul9.Collection;

import java.util.ArrayList;
import java.util.Iterator;
import java.util.List;

public class Collection {
    public static void main(String[] args) {
        List list = new ArrayList();
        list.add(200);
        list.add("Hello");
        list.add(235);
        list.add("Donny");
        // normal loop
        System.out.println("Normal loop");
        for (int i = 0; i < list.size(); i++) {

```

```

    Object o = list.get(i);
    System.out.println(o);
}
// Loop using for-element
System.out.println("for-element loop");
for (Object o : list) {
    System.out.println(o);
}
// Loop using iterator
System.out.println("loop using iterator");
Iterator itr = list.iterator();
while (itr.hasNext()) {
    Object o = itr.next();
    System.out.println(o);
}
// Loop using Lambda Expression
System.out.println("loop using lambda expression");
list.forEach(o -> System.out.println(o));
// Loop using reference
System.out.println("loop using reference");
list.forEach(System.out::println);
}
}

```

Penjelasan:

Collection.java merupakan program yang mendemonstrasikan penggunaan ArrayList serta teknik iterasinya. Untuk penjelasan terkait file ini adalah sebagai berikut:

1. Inisialisasi Objek

Membuat objek ArrayList tanpa spesifikasi tipe data generik, sehingga list tersebut mampu menampung campuran tipe data sekaligus, seperti nilai Integer dan String.

2. Method **main()**

Menambahkan sejumlah data ke dalam list secara berurutan, kemudian mengeksekusi lima teknik iterasi berbeda untuk membaca elemen, yaitu menggunakan normal loop konvensional, for-element (for-each) loop, interface Iterator, Lambda expression, dan operator method reference.

(InterfaceComparator.java)

```

package Modul9.Collection;

import java.util.ArrayList;
import java.util.Collections;
import java.util.Comparator;
import java.util.List;
class SalaryComparator implements Comparator<Employee> {
    @Override
    public int compare(Employee e1, Employee e2) {
        return e1.getSalary() - e2.getSalary();
    }
}

```

```

    }

    public static void main(String[] args) {
        List<Employee> listEmp = new ArrayList();
        listEmp.add(new Employee("bobby", 3000));
        listEmp.add(new Employee("erick", 1600));
        listEmp.add(new Employee("rey", 2500));
        listEmp.add(new Employee("anna", 3500));
        Collections.sort(listEmp);
        System.out.println("Sorted by name");
        listEmp.forEach(System.out::println);
        Collections.sort(listEmp, new SalaryComparator());
        System.out.println("Sorted by salary");
        listEmp.forEach(System.out::println);
    }
}

```

Penjelasan:

InterfaceComparator.java mendemonstrasikan implementasi dari interface Comparator untuk menyediakan aturan pengurutan kustom pada suatu himpunan data. Untuk penjelasan rinci terkait file ini adalah sebagai berikut:

1. Method **compare()** pada class SalaryComparator

Meng-override method bawaan dari interface Comparator untuk membandingkan dua instansiasi objek Employee. Method ini mengembalikan operasi matematika berupa selisih gaji untuk menentukan posisi urutan elemen secara ascending.

2. Method **main()**

Menginisialisasi ArrayList berisi sekumpulan objek Employee. Program melakukan dua tahap pengurutan menggunakan fungsi Collections.sort, pertama berdasarkan abjad bawaan, dan yang kedua disisipkan objek SalaryComparator sebagai argumen tambahan untuk memaksa pengurutan ulang berdasarkan besaran gaji.

(InterfaceComparable.java)

```

package Modul9.Collection;

import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

class Employee implements Comparable<Employee> {
    private String name;
    private int salary;
    public Employee(String name, int salary) {
        this.name = name;
        this.salary = salary;
    }
    public String getName() {
        return name;
    }
}

```

```

    public int getSalary() {
        return salary;
    }
    @Override
    public String toString() {
        return "name=" + name + ", salary=" + salary;
    }
    @Override
    public int compareTo(Employee e) {
        return name.compareTo(e.name);
    }
    public static void main(String[] args) {
        List<Employee> listEmp = new ArrayList();
        listEmp.add(new Employee("bobby", 3000));
        listEmp.add(new Employee("erick", 1600));
        listEmp.add(new Employee("rey", 2500));
        listEmp.add(new Employee("anna", 3500));
        Collections.sort(listEmp);
        listEmp.forEach(System.out::println);
    }
}

```

Penjelasan:

Interface Comparable.java mendemonstrasikan implementasi interface Comparable pada entitas model untuk mendefinisikan aturan pengurutan bawaan. Untuk penjelasan rinci terkait file ini adalah sebagai berikut:

1. Atribut dan Constructor

Terdapat dua atribut private yaitu name bertipe String dan salary bertipe int yang diinisialisasi secara aman melalui constructor menggunakan keyword this.

2. Getter dan Method **toString()**

Menyediakan method getName() dan getSalary() untuk mengakses atribut private dari luar class, serta meng-override method toString() untuk merekonstruksi format cetakan string dari objek.

3. Method **compareTo()**

Meng-override method wajib dari interface Comparable. Method ini membandingkan atribut name antar objek menggunakan fungsi bawaan class String untuk memastikan himpunan data terurut secara leksikografis (abjad).

4. Method **main()**

Menginisialisasi himpunan ArrayList berisi kumpulan objek Employee, kemudian mengeksekusi fungsi Collections.sort() yang secara langsung menerapkan aturan pengurutan bawaan sebelum data dicetak ke layar.

(Filtering.java)

```

package Modul9.Collection;

import java.util.ArrayList;
import java.util.List;
import java.util.stream.Collectors;

class Employee {

```



```

    private String name;
    private int salary;

    public Employee(String name, int salary) {
        this.name = name;
        this.salary = salary;
    }

    public String getName() { return name; }
    public int getSalary() { return salary; }

    @Override
    public String toString() {
        return "Employee{name='" + name + "', salary=" + salary + "}";
    }
}

public class Filtering {
    public static void main(String[] args) {
        List<Employee> listEmp = new ArrayList<>();
        listEmp.add(new Employee("erick", 3500));
        listEmp.add(new Employee("donny", 2500));
        listEmp.add(new Employee("fauzan", 4000));

        System.out.println("Employee with salary minimal 3000");
        for (Employee e : listEmp) {
            if (e.getSalary() >= 3000) {
                System.out.println(e);
            }
        }

        System.out.println("\nEmployee with salary minimal 3000 (Lambda)");
        List<Employee> temp = listEmp.stream()
            .filter(e -> e.getSalary() >= 3000)
            .collect(Collectors.toList());
        temp.forEach(System.out::println);

        System.out.println("\nEmployee erick");
        for (Employee e : listEmp) {
            if (e.getName().equals("erick")) {
                System.out.println(e);
            }
        }

        System.out.println("\nEmployee erick (Lambda)");
        System.out.println(listEmp.stream()
            .filter(e -> e.getName().equals("erick"))
            .findFirst()
            .orElse(null));
    }
}

dompet.bayar(120000);

kartu.bayar(2000000);
kartu.bayar(3500000);

```

```

    rekening.bayar(100000);
    rekening.bayar(200000);

    System.out.println("Cek Saldo & Tagihan:");
    dompet.cekSaldo();
    kartu.cekSaldo();
    rekening.cekSaldo();

    tampilkanRiwayat("Dompet Digital", dompet.getRiwayat());
    tampilkanRiwayat("Kartu Kredit", kartu.getRiwayat());
    tampilkanRiwayat("Rekening Bank", rekening.getRiwayat());
}

private static void tampilkanRiwayat(String judul, List<String> riwayat) {
    System.out.println("\nRiwayat " + judul + ":");
    for (String item : riwayat) {
        System.out.println(item);
    }
}
}

```

Penjelasan:

Filtering.java mendemonstrasikan teknik penyaringan data pada struktur ArrayList menggunakan entitas Employee dan class utama Filtering. Untuk penjelasan rinci terkait file ini adalah sebagai berikut:

1. Class **Employee**

Merepresentasikan entitas data dengan atribut private name dan salary, yang dikelola melalui constructor, method getter, dan override method toString().

2. Method **main()**

Bertindak sebagai titik eksekusi program yang mendemonstrasikan dan membandingkan dua pendekatan penyaringan data dari dalam himpunan ArrayList.

3. Filtering dengan **for-each**

Menyaring data karyawan yang memiliki gaji minimal 3000 atau bernama erick menggunakan proses perulangan for-each konvensional yang dipadukan secara manual dengan struktur logika if bersarang.

4. Pendekatan Stream API

Menyederhanakan proses penyaringan data secara deklaratif menggunakan rantai fungsi .stream() dan .filter(). Data yang tersaring kemudian dikumpulkan kembali menggunakan .collect(), atau diambil presisi data pertamanya secara aman menggunakan fungsi .findFirst().orElse(null).

▪ Hasil Running Program

```
D:\Tugas\SEM4\PBO\PRAKTIKUM>cd "d:\Tugas\SEM4\PBO\PRAKTIKUM\Modul9\VariableCasting\" && javac DownCast.java && java DownCast
Picked up JAVA_TOOL_OPTIONS: -Dstdout.encoding=UTF-8 -Dstderr.encoding=UTF-8
Picked up JAVA_TOOL_OPTIONS: -Dstdout.encoding=UTF-8 -Dstderr.encoding=UTF-8
Error: Could not find or load main class DownCast
Caused by: java.lang.NoClassDefFoundError: DownCast (wrong name: Modul9/VariableCasting/DownCast)
```

Gambar 1. DownCast.java

```
d:\Tugas\SEM4\PBO\PRAKTIKUM\Modul9\VariableCasting>cd "d:\Tugas\SEM4\PBO\PRAKTIKUM\Modul9\VariableCasting\" && javac UpCast.java && java
UpCast
Picked up JAVA_TOOL_OPTIONS: -Dstdout.encoding=UTF-8 -Dstderr.encoding=UTF-8
Picked up JAVA_TOOL_OPTIONS: -Dstdout.encoding=UTF-8 -Dstderr.encoding=UTF-8
Error: Could not find or load main class UpCast
Caused by: java.lang.NoClassDefFoundError: UpCast (wrong name: Modul9/VariableCasting/UpCast)
```

Gambar 2. UpCast.java

```
D:\Tugas\SEM4\PBO\PRAKTIKUM> cmd /C "C:\Users\ACER\AppData\Roaming\Code\User\globalStorage\pleiades.java-extension-pack-jdk\
ava.exe -XX:+ShowCodeDetailsInExceptionMessages -cp C:\Users\ACER\AppData\Roaming\Code\User\workspaceStorage\e9a09083453e4cc
68a2\redhat.java\jdt_ws\PRAKTIKUM_2251628a\bin Modul9.Static.StaticVariable "
44
42
82
422
822
622
```

Gambar 3. StaticVariable.java

```
D:\Tugas\SEM4\PBO\PRAKTIKUM> cmd /C "C:\Users\ACER\AppData\Roaming\Code\User\
ava.exe -XX:+ShowCodeDetailsInExceptionMessages -cp C:\Users\ACER\AppData\Ro
68a2\redhat.java\jdt_ws\PRAKTIKUM_2251628a\bin Modul9.Static.StaticMethod "
5
```

Gambar 4. StaticMethod.java

```

D:\Tugas\SEM4\PBO\PRAKTIKUM> cmd /C "C:\Users\ACER\AppData\Roaming\Code\User\globalStorage
ava.exe -XX:+ShowCodeDetailsInExceptionMessages -cp C:\Users\ACER\AppData\Roaming\Code\User
68a2\redhat.java\jdt_ws\PRAKTIKUM_2251628a\bin Modul9.Collection.Collection "
Normal loop
200
Hello
235
Donny
for-element loop
200
Hello
235
Donny
loop using iterator
200
Hello
235
Donny
Donny
loop using lambda expression
200
Hello
235
Donny
loop using reference
200
Hello
235
Donny

```

Gambar 5. Collection.java

```

D:\Tugas\SEM4\PBO\PRAKTIKUM> cmd /C "C:\Users\ACER\AppData\Roaming\Code\User\globalS
ava.exe -XX:+ShowCodeDetailsInExceptionMessages -cp C:\Users\ACER\AppData\Roaming\Co
68a2\redhat.java\jdt_ws\PRAKTIKUM_2251628a\bin Modul9.Collection.SalaryComparator "
Sorted by name
name=anna, salary=3500
name=bobby, salary=3000
name=erick, salary=1600
name=reyl, salary=2500
Sorted by salary
name=erick, salary=1600
name=reyl, salary=2500
name=bobby, salary=3000
name=anna, salary=3500

```

Gambar 6. InterfaceComparator.java

```

D:\Tugas\SEM4\PBO\PRAKTIKUM> cmd /C "C:\Users\ACER\AppData\Roaming\Code\User
ava.exe -XX:+ShowCodeDetailsInExceptionMessages -cp C:\Users\ACER\AppData\Ro
68a2\redhat.java\jdt_ws\PRAKTIKUM_2251628a\bin Modul9.Collection.Employee "
name=anna, salary=3500
name=bobby, salary=3000
name=erick, salary=1600
name=reyl, salary=2500

```

Gambar 7. InterfaceComparable.java

```
D:\Tugas\SEM4\PBO\PRAKTIKUM> cmd /C "C:\Users\ACER\AppData\Roaming\Code\User\globalStorage\pleiades.ja
ava.exe -XX:+ShowCodeDetailsInExceptionMessages -cp C:\Users\ACER\AppData\Roaming\Code\User\workspace6
68a2\redhat.java\jdt_ws\PRAKTIKUM_2251628a\bin Modul9.Collection.Filtering "
```

Employee with salary minimal 3000
name=erick, salary=3500
name=fauzan, salary=4000

Employee with salary minimal 3000 (Lambda)
name=erick, salary=3500
name=fauzan, salary=4000

Employee erick
name=erick, salary=3500

Employee erick (Lambda)
name=erick, salary=3500

Gambar 8. Filtering.java

- **Link Program Praktikum (Github)**

[kukingkux/PRAKTIKUM-PBO](https://github.com/kukingkux/PRAKTIKUM-PBO)